CH368 FPGA 下位机与上位机开发手册 v2.0

文档版本

- **版本号**: v2.0
- **更新日期**: 2026-04-10
- **适用硬件**: CH368 PCI总线卡 + EP4CE10F17C8 FPGA
- **上位机环境**: Visual Studio 2022 + .NET Framework 4.8

一、系统架构概览

1.1 硬件连接

```
PC主机 (PCI插槽)
    ↓
CH368 PCI总线卡 (CH367芯片)
    ↓ 并行总线
FPGA (EP4CE10F17C8)
    ├─ 编码器输入 (24MHz)
    ├─ 8路相机触发输出
    ├─ 8路光源控制输出
    ├─ 4路气阀控制输出
    ├─ 电机PWM驱动
    ├─ 三色灯/蜂鸣器
    ├─ 振动盘控制
    └─ 下料控制
```

1.2 数据流

```
传感器检测 → 工件入场 → 1024槽位RAM存储
                                ↓
            编码器位置匹配 → 相机触发/气阀动作
                                ↓
                    上位机读取状态/设置参数
```

二、寄存器地址分配表

2.1 全局控制与状态寄存器 (0x00 ~ 0x1F)

地址	R/W	名称	长度	功能描述
0x00	R	magic_num	1B	设备握手标识，恒为 `0x5A`
0x01	R	wp_head_L	1B	工件头指针低8位

地址	R/W	名称	长度	功能描述
0x02	R/W	motor_ctrl	1B	Bit0: 启停(1/0), Bit1: 方向(1/0)
0x03	R/W	motor_speed	1B	电机速度 (0~255)
0x04~0x07	R	global_enc	4B	全局编码器值 (Little-Endian)
0x08~0x0B	R/W	target_reject	4B	气阀触发位置 (Little-Endian)
0x0E	R/W	light_delay	1B	光源延时 (毫秒)
0x0F	R/W	blow_time	1B	吹气时间 (毫秒)
0x10~0x13	R/W	min_spacing	4B	最小工件间距 (防止重叠)
0x14	R/W	sys_ctrl	1B	系统控制寄存器 (见下表)
0x15	R/W	valve_select	1B	气阀选择 (手动测试用)

2.2 系统控制寄存器 (地址 0x14)

Bit	名称	功能
0	freq_strobe	频闪测试模式 (1=启用20Hz频闪)
1	emergency_stop	急停 (1=关闭所有输出)
2	manual_blow	手动吹气测试 (1=启用)
3-7	reserved	保留

2.3 气阀选择寄存器 (地址 0x15)

值	功能
0x00	无动作
0x01	气阀1 (手动测试)
0x02	气阀2 (手动测试)
0x03	气阀3 (手动测试)
0x04	气阀4 (手动测试)

2.4 相机触发位置 (0x20 ~ 0x3F)

地址范围	相机序号	说明
0x20~0x23	相机1	32位触发位置
0x24~0x27	相机2	32位触发位置
0x28~0x2B	相机3	32位触发位置
0x2C~0x2F	相机4	32位触发位置

地址范围	相机序号	说明
0x30~0x33	相机5	32位触发位置
0x34~0x37	相机6	32位触发位置
0x38~0x3B	相机7	32位触发位置
0x3C~0x3F	相机8	32位触发位置

2.5 杂项IO控制 (0x40)

Bit	功能	说明
0	LED红	三色灯-红色
1	LED黄	三色灯-黄色
2	LED绿	三色灯-绿色
3	蜂鸣器	声音报警
4	振动盘	振动盘控制 (新增)
5	下料器	下料控制 (新增)
6-7	reserved	保留

2.6 双翻页器与工件RAM (0x7E ~ 0x87)

地址	R/W	名称	功能描述
0x7E	W	wp_index_H	翻页器高8位
0x7F	W	wp_index_L	翻页器低8位
0x80	R/W	wp_status	工件状态字节
0x81~0x84	R	wp_abs_pos	工件入场绝对位置

三、工件状态字节定义 (地址 0x80)

值	状态	触发动作
0x00	空闲	无工件
0x01	新工件	未判定，等待上位机设置
0x02	1号区	到位后触发气阀1
0x03	2号区	到位后触发气阀2
0x04	3号区	到位后触发气阀3
0x05	4号区	到位后触发气阀4

注意: 旧版本使用 `0xAA`(良品) 和 `0xEE`(次品)，新版本改为 4路气阀选择。

四、引脚分配表

4.1 CH368总线接口 (P2排针)

信号	FPGA引脚	说明
ch368_data[0]	PIN_M8	数据位D0
ch368_data[1]	PIN_L8	数据位D1
ch368_data[2]	PIN_L7	数据位D2
ch368_data[3]	PIN_T6	数据位D3
ch368_data[4]	PIN_R6	数据位D4
ch368_data[5]	PIN_N8	数据位D5
ch368_data[6]	PIN_P8	数据位D6
ch368_data[7]	PIN_K9	数据位D7
ch368_addr[0]	PIN_M9	地址位A0
ch368_addr[1]	PIN_P9	地址位A1
ch368_addr[2]	PIN_N9	地址位A2
ch368_addr[3]	PIN_M10	地址位A3
ch368_addr[4]	PIN_P11	地址位A4
ch368_addr[5]	PIN_N11	地址位A5
ch368_addr[6]	PIN_R7	地址位A6
ch368_addr[7]	PIN_L10	地址位A7
ch368_wr_n	PIN_T9	写使能 (低有效)
ch368_rd_n	PIN_T10	读使能 (低有效)

4.2 光源输出 (P3排针)

信号	FPGA引脚	PCB接口
light_out[0]	PIN_A15	X1
light_out[1]	PIN_F11	X3
light_out[2]	PIN_A12	X5
light_out[3]	PIN_A10	X7
light_out[4]	PIN_D8	X9

信号	FPGA引脚	PCB接口
light_out[5]	PIN_B6	X11
light_out[6]	PIN_C8	预留
light_out[7]	PIN_D6	预留

4.3 相机输出 (P3排针)

信号	FPGA引脚	PCB接口
camera_out[0]	PIN_A14	X2
camera_out[1]	PIN_A13	X4
camera_out[2]	PIN_A11	X6
camera_out[3]	PIN_A9	X8
camera_out[4]	PIN_B7	X10
camera_out[5]	PIN_D11	X12
camera_out[6]	PIN_C7	预留
camera_out[7]	PIN_D7	预留

4.4 气阀输出 (P3排针)

信号	FPGA引脚	PCB接口
valve_out[0]	PIN_E10	X13 (1#气阀)
valve_out[1]	PIN_D9	X14 (2#气阀)
valve_out[2]	PIN_F9	X15 (3#气阀)
valve_out[3]	PIN_E8	X16 (4#气阀)

4.5 电机与编码器

信号	FPGA引脚	PCB接口	说明
motor_pwm_out	PIN_C6	X20	PWM输出
motor_en_out	PIN_D12	OUT6	电机使能
motor_dir_out	PIN_C12	预留	方向控制
enc_a	PIN_E11	LA-LAR	编码器A相
enc_b	PIN_F6	LB-LBR	编码器B相
enc_z	PIN_F7	预留	编码器Z相

4.6 传感器与杂项控制

信号	FPGA引脚	PCB接口	说明
sensor_in	PIN_A7	IN25	传感器输入
w_misc_led_r	PIN_B11	OUT4	三色灯-红
w_misc_led_y	PIN_B10	OUT3	三色灯-黄
w_misc_led_g	PIN_B9	OUT2	三色灯-绿
w_misc_buzzer	PIN_C9	OUT1	蜂鸣器
vibrator_out	PIN_B13	OUT7	振动盘控制
feeder_out	PIN_B14	OUT8	下料控制

4.7 时钟与复位

信号	FPGA引脚	说明
clk_50m	PIN_E1	24MHz系统时钟
rst_n	PIN_E15	复位信号 (低有效)

五、上位机开发指南

5.1 开发环境搭建

5.1.1 软件要求

- Visual Studio 2022
- .NET Framework 4.8
- CH367DLL64.DLL (CH368驱动库)

5.1.2 项目配置

1. 创建 Windows Forms 应用程序
2. 添加 CH367DLL64.DLL 到项目
3. 引用 `System.Runtime.InteropServices` 命名空间

5.2 API函数声明

```
public class CH368_API
{
    const string DLL_PATH = "CH367DLL64.DLL";

    [DllImport(DLL_PATH, CallingConvention = CallingConvention.StdCall)]
    public static extern IntPtr CH367mOpenDevice(int iIndex, bool iEnableMemory,
                                                 bool iEnableInterrupt, byte
iInterruptMode);

    [DllImport(DLL_PATH, CallingConvention = CallingConvention.StdCall)]
```

```
    public static extern void CH367mCloseDevice(int iIndex);

    [DllImport(DLL_PATH, CallingConvention = CallingConvention.StdCall)]
    public static extern bool CH367mGetIoBaseAddr(int iIndex, out IntPtr
oIoBaseAddr);

    [DllImport(DLL_PATH, CallingConvention = CallingConvention.StdCall)]
    public static extern bool CH367mReadIoByte(int iIndex, IntPtr iAddr, out byte
oByte);

    [DllImport(DLL_PATH, CallingConvention = CallingConvention.StdCall)]
    public static extern bool CH367mWriteIoByte(int iIndex, IntPtr iAddr, byte
iByte);
}
```

5.3 初始化流程

```
private IntPtr ioBaseAddr;

// 1. 打开设备
IntPtr handle = CH368_API.CH367mOpenDevice(0, false, false, 0);
if (handle == IntPtr.Zero || handle.ToInt64() == -1)
{
    MessageBox.Show("未检测到 CH368 设备！");
    return;
}

// 2. 获取基地址
if (CH368_API.CH367mGetIoBaseAddr(0, out ioBaseAddr))
{
    // 3. 初始化RAM（清空1024个槽位）
    for (int i = 0; i < 1024; i++)
    {
        CH368_API.CH367mWriteIoByte(0, IntPtr.Add(ioBaseAddr, 0x7E), (byte)((i >>
8) & 0xFF));
        CH368_API.CH367mWriteIoByte(0, IntPtr.Add(ioBaseAddr, 0x7F), (byte)(i &
0xFF));
        CH368_API.CH367mWriteIoByte(0, IntPtr.Add(ioBaseAddr, 0x80), 0x00);
    }
}
```

5.4 寄存器读写操作

5.4.1 写入单字节寄存器

```
void WriteByte(byte addr, byte data)
{
    CH368_API.CH367mWriteIoByte(0, IntPtr.Add(ioBaseAddr, addr), data);
}
```

5.4.2 读取单字节寄存器

```
byte ReadByte(byte addr)
{
    CH368_API.CH367mReadIoByte(0, IntPtr.Add(ioBaseAddr, addr), out byte data);
    return data;
}
```

5.4.3 写入32位寄存器

```
void WriteDWord(byte baseAddr, int value)
{
    CH368_API.CH367mWriteIoByte(0, IntPtr.Add(ioBaseAddr, baseAddr), (byte)(value
& 0xFF));
    CH368_API.CH367mWriteIoByte(0, IntPtr.Add(ioBaseAddr, baseAddr + 1), (byte)
((value >> 8) & 0xFF));
    CH368_API.CH367mWriteIoByte(0, IntPtr.Add(ioBaseAddr, baseAddr + 2), (byte)
((value >> 16) & 0xFF));
    CH368_API.CH367mWriteIoByte(0, IntPtr.Add(ioBaseAddr, baseAddr + 3), (byte)
((value >> 24) & 0xFF));
}
```

5.4.4 读取32位寄存器

```
int ReadDWord(byte baseAddr)
{
    CH368_API.CH367mReadIoByte(0, IntPtr.Add(ioBaseAddr, baseAddr), out byte b0);
    CH368_API.CH367mReadIoByte(0, IntPtr.Add(ioBaseAddr, baseAddr + 1), out byte
b1);
    CH368_API.CH367mReadIoByte(0, IntPtr.Add(ioBaseAddr, baseAddr + 2), out byte
b2);
    CH368_API.CH367mReadIoByte(0, IntPtr.Add(ioBaseAddr, baseAddr + 3), out byte
b3);
    return (b3 << 24) | (b2 << 16) | (b1 << 8) | b0;
}
```

5.5 位掩码操作 (先读后写)

```
void SetBit(byte addr, int bitPos, bool setBit)
{
    CH368_API.CH367mReadIoByte(0, IntPtr.Add(ioBaseAddr, addr), out byte
currentVal);
```

```
    if (setBit)
        currentVal |= (byte)(1 << bitPos);   // 设置位
    else
        currentVal &= (byte)~(1 << bitPos);  // 清除位
    CH368_API.CH367mWriteIoByte(0, IntPtr.Add(ioBaseAddr, addr), currentVal);
}
```

六、功能操作示例

6.1 电机控制

```
// 启动电机
WriteByte(0x02, 0x01);

// 停止电机
WriteByte(0x02, 0x00);

// 设置速度
WriteByte(0x03, 128);
```

6.2 相机位置设置

```
// 设置相机1触发位置为50000
WriteDWord(0x20, 50000);

// 设置相机2触发位置为55000
WriteDWord(0x24, 55000);
```

6.3 气阀控制

```
// 设置吹料位置
WriteDWord(0x08, 20000);

// 设置吹气时间
WriteByte(0x0F, 50);  // 50ms

// 手动测试气阀1
WriteByte(0x15, 0x01);  // 选择气阀1
SetBit(0x14, 2, true);  // 启用手动吹气
```

6.4 工件状态设置

```
void SetWorkpieceStatus(int slotIndex, byte status)
{
    // 1. 定位槽位
    CH368_API.CH367mWriteIoByte(0, IntPtr.Add(ioBaseAddr, 0x7E), (byte)((slotIndex
>> 8) & 0xFF));
    CH368_API.CH367mWriteIoByte(0, IntPtr.Add(ioBaseAddr, 0x7F), (byte)(slotIndex
& 0xFF));

    // 2. 写入状态 (0x02=1号区, 0x03=2号区, 0x04=3号区, 0x05=4号区)
    CH368_API.CH367mWriteIoByte(0, IntPtr.Add(ioBaseAddr, 0x80), status);
}
```

6.5 急停控制

```
// 启用急停
SetBit(0x14, 1, true);

// 解除急停
SetBit(0x14, 1, false);
```

6.6 杂项IO控制

```
// 开启红灯
SetBit(0x40, 0, true);

// 开启蜂鸣器
SetBit(0x40, 3, true);

// 开启振动盘
SetBit(0x40, 4, true);

// 开启下料器
SetBit(0x40, 5, true);
```

七、典型应用场景

7.1 工件分拣流程

```
1. 传感器检测到工件入场 → FPGA自动记录位置到RAM
2. 工件经过相机位置 → FPGA自动触发相机拍照
3. 视觉系统识别工件 → 上位机设置工件状态(1-4号区)
4. 工件到达吹料位置 → FPGA自动触发对应气阀
5. 工件被吹入对应区域 → 完成分拣
```

7.2 多相机协同拍照

```
// 设置8个相机的触发位置
int[] camPositions = { 5000, 6000, 7000, 8000, 9000, 10000, 11000, 12000 };
for (int i = 0; i < 8; i++)
{
    WriteDWord((byte)(0x20 + i * 4), camPositions[i]);
}

// 设置光源延时
WriteByte(0x0E, 2);  // 2ms延时
```

7.3 急停与安全控制

```
// 紧急停止
void EmergencyStop()
{
    SetBit(0x14, 1, true);  // 急停位置1

    // 关闭所有IO
    WriteByte(0x40, 0x00);
}

// 恢复运行
void Resume()
{
    SetBit(0x14, 1, false);  // 解除急停
}
```

八、调试与故障排查

8.1 设备连接问题

症状: 无法打开设备
排查步骤:

1. 检查CH368卡是否插好
2. 检查驱动是否安装
3. 检查设备管理器中是否识别
4. 尝试重新插拔PCI卡

8.2 读写失败

症状: 读写的值不正确
排查步骤:

1. 检查基地址是否获取成功

2. 使用原始读写测试 (地址0x00应返回0x5A)
3. 检查地址是否正确
4. 检查多字节读写的顺序

8.3 气阀不动作

症状: 气阀不触发
排查步骤:

1. 检查工件状态是否设置正确 (0x02-0x05)
2. 检查吹料位置是否正确
3. 检查编码器是否工作
4. 检查是否启用了急停

8.4 相机不触发

症状: 相机不拍照
排查步骤:

1. 检查相机触发位置是否设置
2. 检查编码器是否工作
3. 检查是否启用了急停
4. 检查光源延时设置

九、版本更新记录

v2.0 (2026-04-10)

- ☑ 气阀系统从2路扩展为4路
- ☑ 新增振动盘控制 (地址0x40 Bit4)
- ☑ 新增下料控制 (地址0x40 Bit5)
- ☑ 新增系统控制寄存器 (地址0x14)
- ☑ 新增气阀选择寄存器 (地址0x15)
- ☑ 新增最小工件间距配置 (地址0x10-0x13)
- ☑ 频闪频率提升至20Hz
- ☑ 修复写使能BUG
- ☑ 所有Verilog文件转换为GB2312编码

v1.0 (初始版本)

- 8路相机触发
- 8路光源控制
- 2路气阀控制
- 电机PWM驱动
- 1024工件槽位RAM
- 双翻页器访问机制

十、附录

10.1 时序参数

参数	值	说明
系统时钟	24MHz	FPGA主时钟
频闪频率	20Hz	测试模式频闪
气阀脉宽	可配置	通过0x0F设置
光源延时	可配置	通过0x0E设置

10.2 电气参数

参数	值	说明
IO电平	3.3V LVTTL	所有IO引脚
输出电流	最大24mA	单个IO
编码器频率	最高1MHz	正交编码器

文档结束